

GamerStore ERP

Guía de exposición dividida en 6 partes

Sistema de administración de una tienda de tecnología · Spring Boot (API REST) + React (Vite) + MySQL · Seguridad con JWT

Cómo está armado (lo que todos deben saber en 30 segundos)

El sistema es el **panel de administración (ERP)**. Corre en dos servidores: **Backend** en :8080 (API, rutas /api/**) y **Frontend** del panel en :5173. La base de datos es MySQL (*tienda_pc*). El backend está organizado en **capas**:

Pantalla admin (React) → Controller (/api) → Service (reglas) → Repository → Base de datos
DTO ← Mapper (Entidad→DTO) ←

#	Parte (módulo del ERP)	Responsable	En una frase
1	Arquitectura y modelo de datos	_____	Las capas del sistema y las entidades que se vuelven tablas
2	Datos reales + imágenes	_____	El seeder que llena la BD y las imágenes guardadas en el proyecto
3	Seguridad: login JWT + sesión	_____	Login con token, protección del panel, refresco y contador
4	Productos y Categorías	_____	ABM del catálogo con subida de imágenes
5	Clientes (RENIEC), Pedidos, PDF	_____	DNI real, registrar ventas y descargar el reporte
6	Dashboard, Usuarios, validaciones	_____	Estadísticas, gestión de usuarios y anti-duplicados

💡 **Truco para el Q&A:** todos los módulos siguen el MISMO flujo (**pantalla** → **Controller** → **Service** → **Repository** → **base de datos** y de vuelta). Si te preguntan algo de otra parte, explica ese flujo y di en qué archivo vive; nadie se queda callado.

1

Arquitectura y modelo de datos

Responsable: _____

El cimiento sobre el que trabajan todos: cómo se organiza el código en capas y cómo las clases de Java se convierten en las tablas de la base de datos.

¿Qué hace?

Define la **estructura por capas** del backend, la **conexión a la base de datos** y las **entidades** (clases Java que Hibernate convierte en tablas) junto con sus **repositorios** (acceso a datos sin escribir SQL).

Flujo (cómo funciona)

- 1 `application.properties` define la conexión a MySQL (BD `tienda_pc`) y que Hibernate administre el esquema (`ddl-auto=update`).
- 2 Al arrancar, **Hibernate** lee las clases `@Entity` de `model/` y **crea/actualiza las tablas** automáticamente (no escribimos `CREATE TABLE`).
- 3 Los **repositorios** (interfaces de Spring Data JPA en `repository/`) generan las consultas a partir del nombre del método (ej. `existsByDni`).
- 4 Todo lo demás usa este esqueleto: **Controller** → **Service** → **Repository** → **BD**, y de regreso el **Mapper** convierte la entidad a un **DTO** para enviarlo como JSON.

Carpetas y archivos

- ▶ `pom.xml` — dependencias (Web, Data JPA, Security, MySQL, OpenPDF...)
- ▶ `src/main/resources/application.properties` — conexión a la BD y configuración
- ▶ `.../model/` — entidades: Producto, Categoria, Cliente, Pedido, PedidoItem, Usuario, Rol, RefreshToken
- ▶ `.../repository/` — un repositorio por entidad (Spring Data JPA)
- ▶ `.../GamerStoreApplication.java` — clase de arranque (main)

Qué mostrar en el código

model/Producto.java: `@Entity, @Id @GeneratedValue, @Column(unique=true)` en nombre y `@ManyToOne` a Categoria (relación). · **ProductoRepository.java:** métodos que se convierten en consultas por su nombre.

▶ **Demo en vivo:** abre phpMyAdmin (XAMPP) → BD **tienda_pc** → muestra las tablas (producto, categoria, pedido...) creadas desde las entidades.

? Preguntas típicas + respuesta

¿Quién crea las tablas?

Hibernate (el ORM), a partir de las anotaciones `@Entity`/`@Column`. No escribimos SQL de creación.

¿Qué es un repositorio de Spring Data?

Una interfaz donde declaramos métodos (ej. `findByDni`) y Spring genera la consulta solo, por el nombre del método.

¿Por qué separar en capas?

Cada capa tiene una responsabilidad: Controller recibe la petición, Service pone las reglas, Repository habla con la BD. Es más ordenado y fácil de mantener.

¿Qué es `@ManyToOne`?

Una relación: muchos productos pertenecen a una categoría; en la tabla se guarda `categoria_id` (llave foránea).

2

Datos reales (seeder) + imágenes en el proyecto

Responsable: _____

Cómo el sistema arranca ya "con vida": se llena solo con datos reales de tecnología, y dónde viven las imágenes de los productos.

🎯 ¿Qué hace?

Llena la base de datos al iniciar con **datos reales** (10 categorías, 28 productos de tecnología con precio en S/, ~40 pedidos históricos y clientes con nombres reales de RENIEC) y guarda las **imágenes en el proyecto** (no en la base de datos).

🔄 Flujo (cómo funciona)

- 1 **DataSeeder** implementa `CommandLineRunner` → se ejecuta 1 vez al arrancar el backend.
- 2 Es **idempotente**: cada bloque solo siembra si la tabla está vacía (`count() == 0`), así no duplica al reiniciar.
- 3 Inserta categorías, productos (cada uno con la **ruta** de su imagen y precio en S/), clientes y ~40 pedidos con **fechas repartidas en 6 meses**.
- 4 Las imágenes están en la carpeta del proyecto `uploads/productos/` y se sirven en la URL `/images/**` gracias a **WebConfig** (en desarrollo, Vite las reenvía al backend).

📁 Carpetas y archivos

- ▶ `.../config/DataSeeder.java` — la carga de datos (categorías, productos, clientes, pedidos)
- ▶ `.../config/WebConfig.java` — sirve las imágenes en `/images/**`
- ▶ `uploads/productos/` — las imágenes físicas (dentro del proyecto)
- ▶ `application.properties` — carpeta de subidas y límite de tamaño (multipart)

👁️ Qué mostrar en el código

DataSeeder.java: el `if (count() == 0)` y la lista de productos con su ruta de imagen. · **WebConfig.java**: el *resource handler* que expone `/images/productos/**`.

▶ **Demo en vivo**: muestra la carpeta `uploads/productos/` con las imágenes; abre una en el navegador (`/images/productos/rtx-4070.jpg`); en Pedidos, señala fechas de distintos meses.

? Preguntas típicas + respuesta

¿Por qué las imágenes no van en la base de datos?

Guardar binarios en la BD la hace pesada y lenta; guardamos el **archivo en el proyecto** y en la BD solo la **ruta**.

¿Por qué no se duplican los datos si reinicio?

El seeder solo siembra cuando la tabla está vacía (`count() == 0`).

¿Cómo hay pedidos de meses pasados?

El seeder les asigna una fecha en el pasado (aleatoria pero reproducible), útil para que el dashboard tenga historia.

¿Los nombres de los clientes son reales?

Sí, se obtienen por DNI desde RENIEC (apiperu.dev) al sembrar — el detalle está en la Parte 5.

3

Seguridad: login JWT + refresh + sesión

Responsable: _____

Cómo el ERP protege el acceso: solo entra quien inicia sesión, con un token que se renueva solo.

🎯 ¿Qué hace?

Autenticación **real**: el login entrega un **token JWT** que se exige para todo el panel. El token de acceso dura poco (5 min) y se **renueva solo** con un *refresh token*; hay un **contador de sesión** visible.

🔄 Flujo (cómo funciona)

- 1 En **Login.jsx** el admin envía usuario/clave → `POST /api/auth/login`.
- 2 **AuthController** valida con Spring Security; si es correcto, **JwtService** firma un **access token** y **RefreshTokenService** crea un **refresh** guardado en la BD.
- 3 El front guarda ambos en `localStorage` y en **cada petición** manda `Authorization: Bearer <token>`.
- 4 **JwtAuthenticationFilter** valida el token en cada request; **SecurityConfig** exige rol ADMIN en `/api/admin/**`.
- 5 Si el access vence → **401**; el front llama a `/api/auth/refresh` (que **rota** el refresh) y **reintenta solo**, sin sacar al usuario.
- 6 **SessionTimer.jsx** lee la expiración dentro del token y muestra la cuenta atrás.

📁 Carpetas y archivos

- ▶ `.../config/security/` — `JwtService`, `JwtAuthenticationFilter`, `SecurityConfig`, `CustomUserDetailsService`, `RefreshTokenService`
- ▶ `.../controller/AuthController.java` — login, refresh, logout, me
- ▶ `.../model/RefreshToken.java` + `repository/RefreshTokenRepository.java`
- ▶ `frontend/src/api/client.js` — Bearer + refresco silencioso · `auth/` · `components/admin/SessionTimer.jsx` · `pages/admin/Login.jsx`

📄 Qué mostrar en el código

SecurityConfig.java: la lista `authorizeHttpRequests(...)` con `permitAll()` vs `hasRole("ADMIN")`.
client.js: el bloque donde, ante un 401, llama a *refresh* y reintenta.

► **Demo en vivo**: intenta entrar a `/admin` sin login (te manda al login). Inicia sesión; en DevTools → Application → Local Storage muestra `gs_token/gs_refresh`; al minuto final el contador se pone ámbar y NO te saca.

? Preguntas típicas + respuesta

¿Qué es un JWT?

Un token firmado que lleva el usuario y su rol; el servidor verifica la firma, sin guardar sesión (stateless).

¿Por qué access corto + refresh largo?

Si roban el access, expira en 5 min; el refresh se puede **revocar** en la BD (logout real).

¿Dónde se protege el panel?

En **SecurityConfig**: `/api/admin/**` exige `ROLE_ADMIN`. Sin token válido, responde 401.

¿La contraseña viaja/queda en texto plano?

No: se compara y guarda **hasheada con BCrypt**.

4

Productos y Categorías (catálogo del admin)

Responsable: _____

El corazón del catálogo: crear, editar y borrar productos y categorías, con imágenes que se suben al proyecto.

¿Qué hace?

El **CRUD** (crear/leer/actualizar/eliminar) de **productos** y **categorías** dentro del panel, incluyendo la **subida de la imagen** del producto (se guarda en el proyecto) y la **validación de nombres únicos**.

Flujo (cómo funciona)

- 1 Al abrir la pantalla, **AdminProductos.jsx** pide la lista → `GET /api/admin/productos` → Controller → Service → Repository → tabla `producto` → se pinta la tabla.
- 2 Al subir una imagen → `POST /api/admin/uploads`; **UploadController** valida y la guarda en `uploads/productos/`, y devuelve la **ruta** `/images/productos/xxx.jpg`.
- 3 Al guardar → `POST/PUT /api/admin/productos`; **ProductoService** valida que el **nombre no esté repetido** y guarda (solo la ruta de la imagen, nunca el archivo).
- 4 Categorías es igual, más una regla: no se puede borrar una categoría que **tiene productos**.

Carpetas y archivos

- ▶ `frontend/src/pages/admin/AdminProductos.jsx`, `AdminCategorias.jsx`
- ▶ `.../controller/` — `AdminProductoController`, `AdminCategoriaController`, **`UploadController`**
- ▶ `.../service/` — `ProductoService`, `CategoriaService`

Qué mostrar en el código

ProductoService.crear(...): el `existsByNombreIgnoreCase` que lanza el 409 de duplicado. ·

UploadController.subir(...): cómo arma el nombre con UUID y usa `Files.copy(...)`. ·

CategoriaService.eliminar(...): el bloqueo si tiene productos.

► **Demo en vivo:** Productos → Nuevo → sube una imagen (aparece la vista previa) → guarda; intenta crear otro con el mismo nombre (sale toast rojo). Intenta borrar una categoría con productos (se bloquea).

? Preguntas típicas + respuesta

¿Cómo evitas nombres de producto repetidos?

En ProductoService con existsByNombreIgnoreCase; si existe, devuelve 409 y el front muestra un toast.

¿Cómo evitas que suban un archivo raro?

UploadController valida que el content-type empiece por image/ y limita el tamaño (5 MB); el nombre lo genera el servidor (UUID).

¿Qué pasa al borrar una categoría con productos?

Se bloquea: CategoriaService revisa con existsByCategoriaId antes de eliminar.

¿Buscar/paginar la tabla llama al servidor?

No; lo hace el hook useTableControls en el navegador sobre la lista ya cargada.

5

Clientes (RENIEC), Pedidos y Reporte PDF

Responsable: _____

La parte de ventas: clientes con datos reales por DNI, registrar pedidos y descargar un reporte en PDF.

🎯 ¿Qué hace?

Gestiona **clientes** (con datos reales por **DNI desde RENIEC**), registra **pedidos/ventas** (estilo POS) y genera un **reporte PDF** de pedidos con filtros.

🔄 Flujo (cómo funciona)

- 1 **Cliente:** escribes el DNI y pulsas **"Buscar"** → `GET /api/admin/clientes/reniec/{dni}` → **ReniecService** llama a **apiperu.dev** con el token y autocompleta nombres/apellidos reales. Al guardar, **ClienteService** valida DNI/email únicos.
- 2 **Pedido:** eliges cliente y agregas productos → `POST /api/admin/pedidos`; **PedidoService** arma las líneas, **calcula el total** y guarda todo junto (cascade).
- 3 **Reporte:** botón **"Descargar PDF"** → `GET /api/admin/pedidos/reporte.pdf`; **PedidoReporteService** construye el PDF con **OpenPDF** y el navegador lo descarga.

📁 Carpetas y archivos

- `frontend/src/pages/admin/AdminClientes.jsx`, `AdminPedidos.jsx`
- `.../controller/` — `AdminClienteController (/reniec/{dni})`, `AdminPedidoController (/reporte.pdf)`
- `.../service/` — `ClienteService`, `PedidoService`, **`ReniecService`**, **`PedidoReporteService`**
- `application.properties` — token de apiperu (`app.apidevperu.*`)

👁️ Qué mostrar en el código

ReniecService.consultarDni(...): el llamado con `RestClient`, timeouts y `try/catch best-effort`. · **PedidoService.crear(...)**: el bucle que suma `subtotal`. · **PedidoReporteService.generar(...)**: cómo arma la tabla del PDF.

► **Demo en vivo:** Clientes → Nuevo → escribe **70123456** → "Buscar" (se llenan los nombres reales). Pedidos → registra una venta → "Descargar PDF".

? Preguntas típicas + respuesta

¿De dónde salen los nombres reales?

De la API **apiperu.dev** (RENIEC), consultando por DNI desde ReniecService.

¿Qué pasa si esa API se cae?

Está en *best-effort* con timeouts: si falla, el flujo continúa (el seeder usa nombres de respaldo; el form muestra un aviso).

¿Cómo se calcula el total del pedido?

En PedidoService: por cada ítem $\text{precio} \times \text{cantidad}$ y se suman.

¿Cómo generas el PDF?

Con la librería **OpenPDF** en PedidoReporteService; el endpoint devuelve el archivo como descarga.

6

Dashboard, Usuarios y validaciones

Responsable: _____

El “tablero de control” y los detalles que dejan el sistema pulido: estadísticas, gestión de usuarios y avisos de datos duplicados.

🎯 ¿Qué hace?

Muestra el **Dashboard** con indicadores y “más vendidos”, administra los **Usuarios** del sistema y centraliza el **manejo de errores** para avisar con un **toast** cuando algo está duplicado.

🔄 Flujo (cómo funciona)

- 1 **Dashboard: Dashboard.jsx** pide `GET /api/admin/dashboard`; el Controller junta los KPIs (totales, ventas, stock bajo) y el **top de productos**, y se dibujan con **Recharts**.
- 2 **Usuarios: AdminUsuarios.jsx** hace CRUD contra `/api/admin/usuarios`; **UsuarioService** valida username/email únicos, **hashea la clave con BCrypt** y **bloquea borrar al último admin**.
- 3 **Validaciones + toast**: si se crea algo duplicado, el Service lanza un error → **GlobalExceptionHandler** lo convierte en `{ "error": "..."} → el front muestra toast.error(mensaje).`

📁 Carpetas y archivos

- ▶ `.../controller/` — AdminDashboardController, AdminUsuarioController, **GlobalExceptionHandler**
- ▶ `.../service/UsuarioService.java` (y las validaciones `existsBy...` de las otras entidades)
- ▶ `frontend/src/pages/admin/Dashboard.jsx`, `AdminUsuarios.jsx` · `components/ui/Toast.jsx`

👁 Qué mostrar en el código

GlobalExceptionHandler.java: cada `@ExceptionHandler` y a qué código HTTP mapea (400/401/404/409/413). · **UsuarioService.eliminar(...)**: el `if` que impide borrar el último ADMIN. · **Dashboard.jsx**: cómo consume `data.topProductos`.

▶ **Demo en vivo**: intenta crear una categoría o un usuario que ya existe → sale toast rojo con el mensaje. Muestra el Dashboard con los gráficos y “Más vendidos”.

? Preguntas típicas + respuesta

¿Dónde y cómo evitas duplicados?

En los Service con métodos existsBy...; si existe, lanzo un error que GlobalExceptionHandler transforma en 409 con mensaje.

¿La contraseña se guarda en texto plano?

No, se **hashea con BCrypt** antes de guardar.

¿De dónde salen los “más vendidos”?

De una consulta que agrupa las líneas de pedido por producto y suma cantidades (PedidoRepository.topProductos).

¿Por qué no dejas borrar el último admin?

Para no quedarnos sin acceso al panel; UsuarioService lo revisa con countByRol(ADMIN).



Apéndice — Cómo levantar el ERP

Para el que hace la demo

► Pasos

- 1 **XAMPP:** iniciar **MySQL** (si no existe la BD `tienda_pc`, se crea sola al arrancar).
- 2 **Backend:** en la raíz ejecutar `mvnw.cmd spring-boot:run`. Levanta en `:8080` y siembra los datos.
- 3 **Frontend (panel):** en `frontend/` ejecutar `npm run dev`. Abre `http://localhost:5173/admin`.
- 4 **Login:** usuario **admin123**, contraseña **gamerstore123**.



Estructura del backend (capas)

```
src/main/java/com/gamerstore/app/  
├─ model/          (entidades = tablas)  
├─ repository/     (acceso a datos, Spring Data JPA)  
├─ service/        (reglas de negocio, validaciones)  
├─ controller/     (endpoints /api/...)  
├─ dto/ mapper/    (objetos de transporte + conversión)  
└─ config/         (seeder, seguridad JWT, web)
```



Recuerda: el flujo es siempre **pantalla** → **Controller** → **Service** → **Repository** → **base de datos**. Si dominas eso, puedes explicar cualquier módulo del ERP.